

Numerical Addendum: Biot–Savart Calculator

Generated/Reviewed Documentation (Codex/Opus/Human) for v0.0.1

At the numerical level, the calculator is a closed-form wire field, summed over the polygonal Johnson–Bovey double loop. The surrounding work supplies the ring geometry, selects which atom–ring pairs are evaluated, turns the magnetic field into an outer-product shielding kernel, records local ring-frame coordinates, and projects the result into the shared $T_0/T_1/T_2$ representation. The listings below are distilled from the source — loops and bookkeeping trimmed — but the constants, guards, and arithmetic are the implementation’s.

1 Ring geometry and acceptance

GeometryResult fills `conf.ring_geometries` before this calculator runs. Each aromatic ring contributes ordered vertices v_i in Å, their mean centre c , a unit normal n , and a mean centre-to-vertex radius. The normal is the least-variance direction from an SVD of the centred vertices, with its sign chosen by $(v_1 - v_0) \times (v_2 - v_0)$. The Haigh–Mallion numerical addendum treats that fit in full; here the Biot–Savart code consumes the geometry. The vertex order matters here too: edge i runs from v_i to v_j , with $j = (i + 1) \bmod n$.

```
conf.ring_geometries[ri] =
    protein.RingAt(ri).ComputeGeometry(positions);

double ring_cutoff = cfg("ring_current_spatial_cutoff"); // 15 Å
auto nearby_rings = spatial.RingsWithinRadius(atom_pos, ring_cutoff);

ctx.distance = (atom_pos - geom.center).norm();
ctx.source_extent = 2.0 * geom.radius; // ring diameter
ctx.atom_index = ai;
ctx.source_ring_index = ri;
if (!filters.AcceptAll(ctx)) continue;
```

The spatial index is a KD-tree over ring centres, queried at 15 Å. It only proposes candidates. The filters then reject centre distances < 0.1 Å, distances ≤ 0.5 `source_extent` = `geom.radius`, and atoms that are ring vertices or covalently bonded to one. These are binary gates: a rejected pair does not enter the field sum, and no accepted contribution is reweighted.

The grid-sampling methods use the same double-loop kernel without atom-topology filters. A grid point is skipped when its centre distance is below 0.1 Å, strictly inside the mean ring radius, or greater than 15 Å.

2 One straight segment

WireSegmentField is the numerical core. It takes segment endpoints a, b , field point r , and current I in SI units, then returns B in tesla. With $\ell = b - a$, $d_A = r - a$, and $d_B = r - b$, the code evaluates, with `BIOT_SAVART_PREFACTOR` = 10^{-7} ,

$$B = 10^{-7} I \frac{\ell \times d_A}{|\ell \times d_A|^2} \left(\frac{\ell \cdot d_A}{|d_A|} - \frac{\ell \cdot d_B}{|d_B|} \right).$$

The cross product is the field direction for this segment. It is perpendicular to the plane containing the wire and the field point, so it points around the wire axis rather than along it. Its length is $|\ell|d_\perp$, where $d_\perp$ is the perpendicular distance from r to the line through the segment. The denominator is therefore $(|\ell|d_\perp)^2$, the squared distance factor in the closed form.

The bracket is the finite-length part of the line integral. Each term is the scalar projection of ℓ onto an endpoint sight line: $\ell \cdot \mathbf{d}_A/|\mathbf{d}_A| = |\ell| \cos \alpha_A$ at endpoint $\mathbf{a}$, and $\ell \cdot \mathbf{d}_B/|\mathbf{d}_B| = |\ell| \cos \alpha_B$ at $\mathbf{b}$. Their difference is the endpoint-cosine difference that replaces the integral along the straight segment.

```
Vec3 dl = b_m - a_m;
Vec3 dA = r_m - a_m;
Vec3 dB = r_m - b_m;

double lenA = dA.norm(), lenB = dB.norm();
if (lenA < cfg("biot_savart_wire_endpoint_guard") ||
    lenB < cfg("biot_savart_wire_endpoint_guard")) return Vec3::Zero();

Vec3 cross = dl.cross(dA);
double crossSq = cross.squaredNorm();
if (crossSq < cfg("biot_savart_wire_axis_guard")) return Vec3::Zero();

double scale = BIOT_SAVART_PREFACTOR * I_A / crossSq;
double proj = dl.dot(dA) / lenA - dl.dot(dB) / lenB;
return scale * proj * cross;
```

The two guards are the two ways this closed form degenerates. The endpoint guard is 10^{-25} m, compared with $|\mathbf{d}_A|$ and $|\mathbf{d}_B|$; at an endpoint, one endpoint projection divides by zero. The axis guard is 10^{-70} , compared with $|\ell \times \mathbf{d}_A|^2$ after metre conversion; on the wire axis $d_\perp = 0$, so both the azimuthal direction and the denominator vanish. In either case the segment returns 0, with no limiting extrapolation.

3 The double polygon loop

JohnsonBoveyField builds the ring current from the segment field above. It makes two copies of the ring polygon, one at $+d_t \mathbf{n}$ and one at $-d_t \mathbf{n}$, where $d_t = \text{ring.JohnsonBoveyLobeOffset}()$. The configured offsets are 0.64 Å for PHE, TYR, and TRP6, 0.52 Å for TRP5, 0.50 Å for HIS, HID, and HIE, and 0.60 Å for TRP9. If the vertex count is below three, the function returns 0.

```
int n = static_cast<int>(vertices.size());
if (n < 3) return Vec3::Zero();

Vec3 offset_ang = normal * lobe_offset_ang;
double halfI_A = 0.5 * current_nA * NANOAMPERES_TO_AMPERES;
Vec3 r_m = point_ang * ANGSTROMS_TO_METRES;

for (int i = 0; i < n; ++i) {
    int j = (i + 1) % n;
    Vec3 au = (vertices[i] + offset_ang) * ANGSTROMS_TO_METRES;
    Vec3 bu = (vertices[j] + offset_ang) * ANGSTROMS_TO_METRES;
    Vec3 al = (vertices[i] - offset_ang) * ANGSTROMS_TO_METRES;
    Vec3 bl = (vertices[j] - offset_ang) * ANGSTROMS_TO_METRES;
    B += WireSegmentField(au, bu, halfI_A, r_m);
    B += WireSegmentField(al, bl, halfI_A, r_m);
}
```

Each lobe carries half the requested current, so the current conversion 10^{-9} A/nA happens once in halfI_A. Positions cross the boundary once as well: 10^{-10} m/Å is applied to the field point and to each segment endpoint. After those conversions, WireSegmentField sees only metres, amperes, and tesla. The 10^6 ppm scale does not enter this field calculation; it reappears when B becomes a shielding kernel.

The loop is the polygon through the ring vertices. A five-, six-, or nine-membered ring contributes five, six, or nine straight segments per lobe, with the last edge wrapping to the first vertex. The segment formula is exact for those straight chords. The approximation is the replacement of the smooth ring-current path by that polygonal path, not a quadrature over the loop.

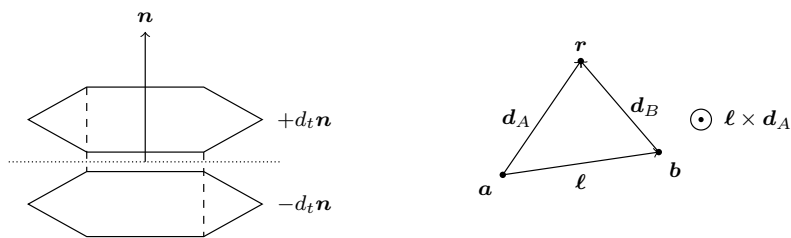


Figure 1: The Johnson–Bovey field is the sum of straight-wire fields over two offset polygons. One segment uses $\ell = b - a$ and endpoint vectors d_A, d_B ; $\ell \times d_A$ points in the segment’s azimuthal field direction.

4 The shielding kernel

For atom calculations the double-loop field is always called with `current_nA = 1.0`. The code records `ring.Intensity()` for provenance, but `Compute()` does not multiply by it. This unit-current convention separates the geometric kernel from the fitted or literature ring-current intensity: B is the field of a 1 nA Johnson–Bovey loop at the accepted atom position.

$$G_{ab} = -n_b B_a \text{ PPM_FACTOR}, \quad \text{PPM_FACTOR} = 10^6.$$

```
Vec3 B = JohnsonBoveyField(geom.vertices, geom.normal,
                          ring.JohnsonBoveyLobeOffset(), 1.0, atom_pos);

Mat3 G;
for (int a = 0; a < 3; ++a)
  for (int b = 0; b < 3; ++b)
    G(a,b) = -geom.normal(b) * B(a) * PPM_FACTOR;

rn->G_tensor = G;
rn->G_spherical = SphericalTensor::Decompose(G);
rn->B_field = B;
G_total += G;
B_total += B;
```

The row index a is the secondary-field component, and the column index b is the applied-field component. The matrix is the outer product $-Bn^T$, scaled by 10^6 , so its rank is at most one. It is generally asymmetric because B need not be parallel to n . The trace is $-10^6 n \cdot B$, hence $T_0 = -10^6 (n \cdot B)/3$, while the antisymmetric and symmetric traceless parts can also be populated.

`WriteFeatures()` exports the per-atom fields `Compute()` wrote. `bs_shielding.npy` is $(N, 9)$, the decomposition of G summed over all rings at the atom, packed in T_0, T_1, T_2 order. The per-type arrays `bs_per_type_T0.npy` and `bs_per_type_T2.npy` are $(N, 8)$ and $(N, 40)$, eight aromatic ring-type slots carrying one T_0 and five T_2 components each. The remaining two are `bs_total_B.npy` at $(N, 3)$ and `bs_ring_counts.npy` at $(N, 4)$ for the $\leq 3, \leq 5, \leq 8, \leq 12$ Å shells. The per-type accumulation stores T_0 and T_2 only; the full $(N, 9)$ channel keeps T_1 .

5 The ring-frame record

The ring calculators share `RingNeighbourhood` records. When this calculator creates the record for an accepted atom–ring pair, it stores a local ring-frame description before writing the Biot–Savart fields. With $d = r - c$, the axial coordinate is $z = d \cdot n$, the in-plane vector is $d_\perp = d - zn$, its length is $\rho = |d_\perp|$, and the folded polar angle is $\theta = \text{atan2}(\rho, |z|)$. The fold makes θ the same above and below the ring; z keeps the side sign.

```

Vec3 d = atom_pos - geom.center;
double z = d.dot(geom.normal);
Vec3 d_plane = d - z * geom.normal;
double rho = d_plane.norm();
double theta = std::atan2(rho, std::abs(z));

Vec3 ref = geom.vertices[0] - geom.center;
Vec3 ref_plane = ref - ref.dot(geom.normal) * geom.normal;
double ref_norm = ref_plane.norm();
double rho_guard = cfg("near_zero_vector_norm_threshold"); // 1e-10

if (rho > rho_guard && ref_norm > rho_guard) {
    Vec3 d_hat = d_plane / rho;
    Vec3 ref_hat = ref_plane / ref_norm;
    rn->cos_phi = d_hat.dot(ref_hat);
    rn->sin_phi = d_hat.cross(ref_hat).dot(geom.normal);
}

```

The azimuth reference is the projected centre-to-vertex-zero direction. The stored cosine gives the cosine of the opening angle between that reference and the atom's in-plane direction, but the cosine alone cannot distinguish the two sides of that reference line. The stored sine, $(\hat{d}_\perp \times \hat{r}_0) \cdot \mathbf{n}$, changes sign across that line under the oriented ring normal. That sign distinguishes the two in-plane sides of an asymmetric ring, such as the nitrogen side and carbon side of an imidazole. If either in-plane vector has norm at most 10^{-10} , the normalization is skipped and the default $(\cos_phi, \sin_phi) = (1, 0)$ remains.

The magnetic field is projected into the same frame. The radial unit vector is formed only when $\rho > 10^{-10}$; otherwise it is left at 0. The stored `B_cylindrical` vector is $(B_\rho, 0, B_z) = (\mathbf{B} \cdot \hat{\rho}, 0, \mathbf{B} \cdot \mathbf{n})$. The azimuthal field component is not stored in this summary.

6 The shared tensor projection

Every 3×3 tensor emitted by this calculator passes through `SphericalTensor::Decompose`. The projection is closed-form arithmetic, not an eigenproblem: the trace gives T_0 , the antisymmetric part gives the three T_1 components, and the symmetric traceless part gives the five T_2 components.

```

st.T0 = sigma.trace() / 3.0;

st.T1[0] = 0.5*(sigma(1,2) - sigma(2,1));
st.T1[1] = 0.5*(sigma(2,0) - sigma(0,2));
st.T1[2] = 0.5*(sigma(0,1) - sigma(1,0));

double Sxx = sigma(0,0) - st.T0;
double Syy = sigma(1,1) - st.T0;
double Szz = sigma(2,2) - st.T0;
double Sxy = 0.5*(sigma(0,1) + sigma(1,0));
double Sxz = 0.5*(sigma(0,2) + sigma(2,0));
double Syz = 0.5*(sigma(1,2) + sigma(2,1));

st.T2[0] = std::sqrt(2.0) * Sxy;
st.T2[1] = std::sqrt(2.0) * Syz;
st.T2[2] = std::sqrt(3.0 / 2.0) * Szz;
st.T2[3] = std::sqrt(2.0) * Sxz;
st.T2[4] = (Sxx - Syy) / std::sqrt(2.0);

```

The T_2 packing is isometric: the five-vector length equals the Frobenius length of the symmetric traceless matrix S . The off-diagonal $\sqrt{2}$ factors pay for double counting, since S_{xy} appears as both S_{xy} and S_{yx} in the matrix. The diagonal pair $\sqrt{3/2} S_{zz}$ and $(S_{xx} - S_{yy})/\sqrt{2}$ carries the two remaining degrees of freedom after $S_{xx} + S_{yy} + S_{zz} = 0$. A dot product of stored T_2 five-vectors is therefore the Frobenius inner product of the symmetric traceless matrices they encode. For the Biot–Savart kernel, all three channels can be populated

because $G = -10^6 \mathbf{B} \mathbf{n}^\top$ is generally asymmetric.

7 Glossary

Finite-wire Biot–Savart field. A straight current segment curls the magnetic field around its axis, and the endpoint sight lines set how much of that curl reaches the field point (an azimuthal field, scaled by a finite-length factor). One edge of a six-membered ring is one such segment; its $\ell \times \mathbf{d}_A$ fixes the azimuthal direction and its two endpoint projection terms give the finite-length cosine difference. Formally, it is the analytic line integral of $\mu_0 I d\ell \times \mathbf{R} / (4\pi |\mathbf{R}|^3)$ over a straight segment; the implementation uses 10^{-7} for $\mu_0 / (4\pi)$.

Johnson–Bovey double loop. Two displaced copies of the ring polygon, one above and one below the fitted ring plane, model the two off-plane π -electron sheets. For a PHE ring those copies sit at $\pm 0.64 \text{ \AA} \mathbf{n}$, and each carries half of the requested current. Formally, the field is the sum of the closed-form finite-wire field over all polygon edges of both displaced loops.

Rank-at-most-one outer product. A column vector times a row vector can change only along one input direction, and it becomes the zero matrix if either vector is zero. Here $G = -10^6 \mathbf{B} \mathbf{n}^\top$, so a field component B_a fills row a in proportion to the normal components n_b . Formally, $\text{rank}(\mathbf{u} \mathbf{v}^\top) \leq 1$ for all $\mathbf{u}, \mathbf{v}$.

Spherical tensor decomposition, $T_0/T_1/T_2$. A 3×3 tensor holds three kinds of content that rotate without mixing — an orientation-free size (the scalar trace, T_0), an axial twist from its antisymmetric part (the three T_1 components), and a five-number traceless shape for the rest of the direction-dependence (the five T_2 components). For one ring's kernel here T_0 is $-10^6 (\mathbf{n} \cdot \mathbf{B})/3$, while T_1 records the antisymmetry when $\mathbf{B}$ and $\mathbf{n}$ are not parallel. Formally, it is the closed-form split of a 3×3 tensor's nine components into dimensions 1, 3, and 5, with the T_2 basis normalized to preserve the Frobenius norm.

Ring-frame coordinates. The atom position is split into height above the ring and in-plane distance from the centre. A point over an imidazole's nitrogen side and one over its carbon side can have the same ρ and z , but opposite stored signed azimuth. Formally, the frame stores $(z, \rho, \theta, \cos \phi, \sin \phi)$ with $z = \mathbf{d} \cdot \mathbf{n}$, $\rho = |\mathbf{d} - z\mathbf{n}|$, $\cos \phi = \hat{\mathbf{d}}_\perp \cdot \hat{\mathbf{r}}_0$, and $\sin \phi = (\hat{\mathbf{d}}_\perp \times \hat{\mathbf{r}}_0) \cdot \mathbf{n}$.